

Update a file through a Python algorithm

Project description

At White Space Digital & Design, I'm building out my cybersecurity portfolio by automating a common access-control task: maintaining an allow list of IP addresses permitted to reach restricted content. The “allow_list.txt” file identifies these approved IP addresses. A separate remove_list identifies IP addresses that should no longer have access. I created an algorithm in Python to automate updating the “allow_list.txt” file by removing any IP addresses that appear on the remove_list, so access is revoked without anyone having to manually edit the file by hand.

Open the file that contains the allow list

For the first part of the algorithm, I opened the “allow_list.txt” file. First, I assigned this file name as a string to the import_file variable:

```
# Assign `import_file` to the name of the file

import_file = "allow_list.txt"
```

Then, I used a with statement to open the file:

```
# Build `with` statement to read in the initial contents of the file

with open(import_file, "r") as file:
```

In my algorithm, the with statement is used with the .open() function in read mode to open the allow list file for the purpose of reading it. The purpose of opening the file is to allow me to access the IP addresses stored in the allow list file. The with keyword helps manage resources by closing the file automatically after exiting the with statement. In the code with open(import_file, "r") as file:, the open() function takes two parameters. The first identifies the file to import, and the second indicates what I want to do with the file — here, "r" indicates that I want to read it. The code also uses the as keyword to assign a variable named file, which stores the output of the .open() function while I work within the with statement.

Read the file contents

In order to read the file contents, I used the .read() method to convert it into a string.

```
with open(import_file, "r") as file:
    # Use `.read()` to read the imported file and store it in a variable named
    `ip_addresses`

    ip_addresses = file.read()
```

When using an .open() function that includes the argument "r" for “read,” I can call the .read() function in the body of the with statement. The .read() method converts the file into a string and allows me to read it. I applied .read() to the file variable identified in the with statement, then assigned the string output to the variable ip_addresses.

In summary, this code reads the contents of the “allow_list.txt” file into a string format that allows me to later organize and extract data from it in my Python program.

Convert the string into a list

In order to remove individual IP addresses from the allow list, I needed it to be in list format. Therefore, I next used the `.split()` method to convert the `ip_addresses` string into a list:

```
# Use `.split()` to convert `ip_addresses` from a string to a list
ip_addresses = ip_addresses.split()
```

The `.split()` function is called by appending it to a string variable, and it works by converting the contents of a string into a list. The purpose of splitting `ip_addresses` into a list is to make it easier to remove individual IP addresses from the allow list. By default, `.split()` splits the text by whitespace (in this case, each IP address sits on its own line) into list elements. In this algorithm, `.split()` takes the data stored in `ip_addresses` and converts it into a list of IP addresses, which I reassigned back to the `ip_addresses` variable.

Iterate through the remove list

A key part of my algorithm involves iterating through the IP addresses that are elements of the `remove_list`. To do this, I incorporated a for loop:

```
# Build iterative statement
# Name loop variable `element`
# Loop through `remove_list`
for element in remove_list:
```

The for loop in Python repeats code for a specified sequence. The overall purpose of the for loop in this algorithm is to apply the same code statements to every element in the `remove_list`. The for keyword starts the loop, followed by the loop variable `element` and the keyword `in`. The `in` keyword indicates that I want to iterate through the sequence `remove_list` and assign each value in turn to the loop variable `element`.

Remove IP addresses that are on the remove list

My algorithm requires removing any IP address from the allow list, `ip_addresses`, that is also contained in `remove_list`. Because there were no duplicates in `ip_addresses`, I was able to use the following code to do this:

```
for element in remove_list:
    # Create conditional statement to evaluate if `element` is in `ip_addresses`

    if element in ip_addresses:
        # use the `.remove()` method to remove
        # elements from `ip_addresses`

        ip_addresses.remove(element)
```

First, within my for loop, I created a conditional that evaluated whether or not the loop variable `element` was found in the `ip_addresses` list. I did this because applying `.remove()` to an element that isn't in `ip_addresses` would raise an error. Then, within that conditional, I applied `.remove()` to `ip_addresses`, passing in the loop variable `element` as the argument, so that each IP address found in the `remove_list` would be removed from `ip_addresses`. This approach relies on `ip_addresses` having no duplicate entries — since `.remove()` only deletes the first matching value it finds, duplicates could leave a removed IP address still present in the list.

Update the file with the revised list of IP addresses

As a final step in my algorithm, I needed to update the allow list file with the revised list of IP addresses. To do so, I first needed to convert the list back into a string. I used the `.join()` method for this:

```
# Convert `ip_addresses` back to a string so that it can be written into the text file
ip_addresses = "\n".join(ip_addresses)
```

The `.join()` method combines all items in an iterable into a single string. It's applied to a string containing the characters that will separate the elements once joined. In this algorithm, I used `.join()` to create a string from the list `ip_addresses` so I could pass it as an argument to the `.write()` method when writing to “allow_list.txt”. I used the string (“\n”) as the separator, so that each IP address would be placed on its own new line, matching the original file's layout.

Then, I used another `with` statement and the `.write()` method to update the file:

```
# Build `with` statement to rewrite the original file

with open(import_file, "w") as file:
    # Rewrite the file, replacing its contents with `ip_addresses`

    file.write(ip_addresses)
```

This time, I used a second argument of “w” with the `open()` function in my `with` statement. This argument indicates that I want to open the file to write over its existing contents. When using “w”, I can call the `.write()` function in the body of the `with` statement; `.write()` writes string data to the specified file and replaces any existing content there. In this case, I wanted to write the updated allow list as a string back to “allow_list.txt” so that the restricted content would no longer be accessible to any IP addresses that had been removed. To rewrite the file, I appended `.write()` to the file object identified in the `with` statement, and passed in the `ip_addresses` variable as the argument.

Summary

I created an algorithm that removes IP addresses identified in a `remove_list` variable from the “allow_list.txt” file of approved IP addresses. This algorithm involved opening the file, converting it to a string so it could be read, and then converting that string into a list stored in the variable `ip_addresses`. I then iterated through the IP addresses in `remove_list`, and with each iteration, evaluated whether the element was part of the `ip_addresses` list. If it was, I applied the `.remove()` method to delete that element from `ip_addresses`. After the loop finished, I used the `.join()` method to convert `ip_addresses` back into a string so I could write over the contents of “allow_list.txt” with the revised, updated list of approved IP addresses.